

P03 Library Management System

Overview

This assignment involves the implementation and testing of a simplified **Library Management System**. The goal of this program is to simulate the core functionalities of a library by distinguishing between two types of users: **subscribers** and **librarians**. The system will allow subscribers to browse, borrow, and return books, while librarians will manage the library's inventory and assist subscribers in checking out or returning books.

The program is expected to handle a variety of **real-world scenarios**, including invalid inputs, errors, and edge cases, by implementing proper **exception handling** and ensuring robust input validation. This will ensure the system operates smoothly and gracefully recovers from unexpected situations, such as invalid book IDs, unavailable books, or actions attempted by unauthorized users.

Learning Objectives

The goals of this assignment include:

- **Enhance Program Robustness:** Learn how to improve the robustness of a program by designing it to handle unusual circumstances and erroneous input gracefully, ensuring it does not crash during unexpected scenarios.
- **Understand Exceptions:** Deepen your understanding of the difference between **checked** and **unchecked exceptions**, and gain practical experience in both throwing and catching exceptions of each kind.
- **Work with Objects:** Gain additional practice in creating and using **objects**.
- **Work with ArrayLists:** Gain additional practice using ArrayList in Java.
- **Test Exception Handling:** Strengthen your testing skills by writing tests that specifically verify whether exceptions are thrown under prescribed circumstances and ensure they are handled appropriately.

Grading Rubric

5 points	Pre-Assignment Quiz: The P03 pre-assignment quiz is accessible through Canvas before having access to this specification by 9:59PM CT on Friday February 07 .
20 points	Immediate Automated Tests: Upon every submission of your assignment to Gradescope, you will receive immediate feedback from automated grading tests about whether specific parts of your submission conform to this write-up specification. If these tests detect problems in your code, they will attempt to give you some feedback about the kind of defect that they noticed. Passing all immediate automated tests does NOT guarantee full credit for the assignment.
15 points	 Additional Automated Tests: The results of these tests will NOT be available immediately after each submission. Instead, the feedback from the Additional Automated Tests will be published on Gradescope only after the assignment deadline , alongside your manual grading feedback. These tests are similar to the Immediate Automated Tests but may evaluate different aspects of your submission or test your code in ways not covered by the Immediate Tests. It is important to note that you will NOT have the opportunity to make up lost points or fix any bugs detected by the additional automated tests for credit after the assignment deadline. Please ensure that you thoroughly test your submission on your own before the deadline to avoid missing points on these tests.
10 points	Manual Grading: Human graders will review the commenting, style, and organization of your final submission. They will be checking whether it conforms to the requirements of the CS300 Course Style Guide . You will NOT be able to resubmit corrections for extra points, and should therefore carefully review the readability of your code with respect to the course style guide.
50 points	MAXIMUM Total Score

Assignment Requirements and Notes

(Please read carefully!)

Pair Programming and Use of External Libraries and Sources

- Pair programming is **NOT ALLOWED** for this assignment.

- Any source code including javadoc style comments provided in this specification may be included verbatim in your program without attribution.
- All other sources must be cited explicitly in your program comments, in accordance with the [Appropriate Academic Conduct guidelines](#).
- Any use of **ChatGPT** or **other large language models (LLM)** must be cited AND your submission **MUST** include a file called `log.txt` containing the full transcript of your usage of the tool. Failure to cite or include your logs is considered academic misconduct and will be handled accordingly.
- You are only allowed to **import** or **use** the libraries listed below in their respective files **only**.
- **The ONLY external libraries** you are permitted to use in your submitted `ExceptionalLibrary.java` and `ExceptionalLibraryTester.java` files are:

```
import java.text.ParseException;
import java.util.ArrayList;
import java.util.NoSuchElementException;
import java.io.File;
import java.io.FileNotFoundException;
import java.io.IOException;
import java.io.PrintWriter;
import java.util.Scanner;
```

- You are also allowed to import the `java.util.Arrays` in your tester file `ExceptionalLibraryTester.java` ONLY.

Important Note on Exception Handling:

- When implementing methods in this assignment, **only throw exceptions that are explicitly declared** in the details of the Javadoc method header using the throws annotation. **No additional exceptions** should be thrown beyond those specified.
- If a method is not expected to throw any exceptions, ensure that you properly handle potential errors within the method instead of allowing unlisted exceptions to propagate. This may involve using appropriate try-catch blocks or validating inputs before performing operations.
- If a method is expected to throw an exception without being caught, let the exception propagate.

CS300 Assignment Requirements

This section is **VALID** for **ALL** the CS300 assignments

- If you need assistance, please check the list of our [Resources](#).
- Read carefully through the specification provided in this write-up. This assignment requires clear understanding of its instructions. Read **TWICE** the instructions and do not hesitate to ask for clarification on piazza if you find any ambiguity.
- You **MUST NOT** add any additional fields either instance or static, and any public methods either static or instance to your program, other than those defined in this write-up.
- You **CAN** define local variables (declared inside a method's body) that you may need to implement the methods defined in this program.
- You **MUST NOT** add any additional fields either instance or static to your program, and any public methods either static or instance to your program, other than those defined in this write-up.
- You **CAN** define **private** methods to help implement the different public methods defined in this program, if needed.
- Your assignment submission must conform to the [CS300 Course Style Guide](#). Please review **ALL** the commenting, naming, and style requirements.
- If starter code files to download are provided, be sure to remove the comments including the **TODO** tags from your last submission to gradescope.
- Avoid submitting code which does not compile. Make sure that **ALL** of your submitted files **ALWAYS** compile. A submission which contains compile errors won't pass any of the automated tests on gradescope.
- You can submit your **work in progress (incomplete work) multiple times** on gradescope. Be sure to include method stubs for the incomplete methods (this includes complete method signature and a default return statement if the method return type is not void). Your submission may include methods not implemented or with partial implementation or with a default return statement.
- Run your program locally before you submit to Gradescope. If it doesn't work on your computer, it will not work on Gradescope.
- You are responsible for maintaining secure back-ups of your progress as you work. The OneDrive and GoogleDrive accounts associated with your UW NetID are often convenient and secure places to store such backups. Aspiring students may try their hands at [version control](#).

- Be sure to submit your code (work in progress) of this assignment on [Gradescope](#) both early and often. This will 1) give you time before the deadline to fix any defects that are detected by the tests, 2) provide you with an additional backup of your work, and 3) help track your progress through the implementation of the assignment. These tests are designed to detect and provide feedback about only very specific kinds of defects. **It is your responsibility to implement additional testing to verify that the rest of your code is functioning in accordance with this write-up.**

1 Getting Started

1. [Create a new project](#) in Eclipse, and name it something like “P03 Library Management System”.
 - a. Ensure this project uses Java 17. Select “JavaSE-17” under “Use an execution environment JRE” in the New Java Project dialog box.
 - b. Do **not** create a project-specific package; use the default package.
2. Download three (3) Java source files from the assignment page on Canvas:
 - `Book.java`
 - `Subscriber.java`
 - `ExceptionalLibraryTester.java`
3. Create one (1) Java source file within your project’s src folder:
 - `ExceptionalLibrary.java`
 - Only the `ExceptionalLibraryTester` class should contain a `main()` method.
 - Do **NOT** submit `Book.java` and `Subscriber.java` to Gradescope.

In this assignment, you will implement a Library Management System that enables book borrowing, returning, and inventory management.

To help you complete this assignment, two essential classes (`Book` and `Subscriber`) are fully provided. Your task is to implement the `ExceptionalLibrary` class (which handles the core library functionalities) and a tester class `ExceptionalLibraryTester` to verify the correctness of the implemented methods.

What to Do Next:

- **Read through the `Book` and `Subscriber` classes carefully.**
- **Plan your implementation** of `ExceptionalLibrary`, ensuring proper exception handling.
- **Develop and test** your methods using `ExceptionalLibraryTester`.

2 Read through the Provided Classes: Book and Subscriber

These classes define the fundamental data structures for the library system. Before beginning your implementation, carefully read through the implementation details of these classes and understand how they function.

2.1 Book (Instantiable Class)

- This class models a **book** in the library's inventory.
- Each book has a **unique identifier (ID)**, a **title**, an **author**, and a **borrower card barcode** (indicating who has borrowed it).
- This class supports the following operations:
 - **Check** if the **book** is **available** (`isAvailable()` method).
 - **Borrow** the book (`borrowBook(int borrowerCardBarCode)`).
 - **Return** the book (`returnBook()`).
 - **Retrieve** the book's details (`getTitle()`, `getAuthor()`, `getID()`).
 - **Get** a **formatted string representation** of the book (`toString()`).
- Note that the **book ID** is automatically assigned and increments sequentially.
- The **borrowerCardBarCode** is null when the book is available.
- The `isAvailable()` method checks whether a book is currently borrowed.

2.2 Subscriber (Instantiable Class)

- This class models a **library subscriber**, who can borrow and return books.
- Each subscriber has a **unique library card barcode**, a **PIN**, a **name**, and an **address**.
- This class supports the following operations:
 - **Check out** books (`checkoutBook(Book book)`).
 - **Return** books (`returnBook(Book book)`).
 - Check **borrowing history** (`getBorrowedBooks()`, `getReturnedBooks()`).
 - Verify if this subscriber **has checked out** a **specific book** and has not yet returned it, (`hasCheckedOut(Book book)`).
 - **Retrieve/Get** this subscriber's details (`getName()`, `getCARD_BAR_CODE()`).
- **Note** that a **subscriber can borrow up to MAX_BOOKS_CHECKED_OUT** books at a time.

- The **library card barcode** is automatically assigned (starting from 2025000001).
- Each subscriber has a **borrowing history**, tracking all returned books.

3 Implement the ExceptionalLibrary and ExceptionalLibraryTester Classes

Now that you understand the `Book` and `Subscriber` classes, you need to implement the `ExceptionalLibrary` class in its entirety in accordance to the details provided in these [JavaDocs](#).

Test-Driven Implementation Approach (Recommended)

The `ExceptionalLibraryTester.java` provided file contains stub-implementations for all of the `REQUIRED` test methods to implement. We encourage you to write additional tester methods to check the correctness of other methods. Any additional tester methods must be `private static`.

Before implementing the methods in `ExceptionalLibrary`, first write tester methods in the tester class that cover a variety of test scenarios. This Test-Driven Development (TDD) approach will help you:

- Define expected behavior before writing the actual implementation.
- Identify edge cases early and ensure robust exception handling.
- Debug and refine your implementation more efficiently.

3.1 ExceptionalLibrary Class

The `ExceptionalLibrary` class serves as the **core** of the Library Management System. Since we have not yet covered creating instantiable classes, you are going to implement this class as a **Utility** class. This utility class provides **static methods** to manage a collection of books and subscribers, ensuring proper handling of various operations while incorporating **exception handling** for robustness. This includes:

- **Adding and removing books** from the inventory.
- **Managing subscribers** (adding new subscribers, finding existing ones).
- **Handling book checkouts and returns**.
- **Searching for books** by title, author, or ID.
- **Ensuring proper exception handling** for invalid inputs and exceptional circumstances.

ExceptionalLibrary Static Fields:

Below is the list of **static fields** defined in the `ExceptionalLibrary` class. These fields **must be declared outside of any method**. A common practice is to define them at **the top of the class**, before the method implementations. Be sure to add **descriptive comments** on each of these fields.

```
private static ArrayList<Book> books = new ArrayList<>();  
private static ArrayList<Subscriber> subscribers = new ArrayList<>();
```

Note that in this assignment, the *books* and the *subscribers* `ArrayLists` are initialized when they are declared.

ExceptionalLibrary Static Methods:

Below is the list of methods you must implement in the `ExceptionalLibrary` class. Further details are provided in these [JavaDocs](#).

Before you begin implementation, add method stubs to your class, ensuring each has a complete method header in accordance to these [JavaDocs](#) and, if necessary, a default return statement.

1. Library Initialization and Information Retrieval

- `resetLibrarySystem()`: Resets the library system by creating empty lists for books and subscribers, while also resetting book identifiers.
- `getBooks()`: Returns a perfect-size array storing the books present in the list of books in the same order.
- `getSubscribers()`: Returns a perfect-size array storing the subscribers present in the list of subscribers in the same order.
- `getLibraryInventoryAsString()`: Returns a `String` representation of each book in the library inventory (list of books).

2. Managing Books

- `addBook(String title, String author)`: Adds a new book to the library inventory.
- `removeBook(int bookId)`: Removes a book from the library if it is available; throws an exception if the book is checked out. A book currently borrowed by a subscriber cannot be removed until returned.
- `findBookById(int bookId)`: Searches for a book by its unique identifier, throws an exception if book not found.
- `findBookByAuthor(String author)`: Retrieves a list of books by a specific author (case-insensitive).

3. Managing Subscribers

- `addSubscriber(String name, String pin, String address)`: Registers a new subscriber and assigns a library card. **Error Handling by Throwing Specific Exceptions:** Ensures that new subscribers provide valid information like name, address, and 4-digit pin code and handle the scenarios where new library cards cannot issued.
- `findSubscriber(int cardBarCode)`: Finds a subscriber by their library card barcode.

4. Exception Handling and Input Validation

The class includes built-in error handling to ensure robustness.

- `parsePinCode(String s)`: Converts a string into a **valid four-digit PIN**; throws a `ParseException` if the input is invalid. We recommend implementing this method **before** implementing `addSubscriber`.

Important Note:

Note carefully that the `Integer.parseInt(String)` throws a `NumberFormatException` if the provided string does not contain a parsable integer. However, your `parsePinCode(String s)` **MUST** throw a `ParseException` (NOT a `NumberFormatException`) if the provided `String` argument `s` cannot be parsed correctly.

Below is an example of how to throw a new `ParseException` object.

```
throw new ParseException("error message", errorOffset);
```

You can set the `errorOffset` to zero 0.

- Methods like `addBook()`, `removeBook()`, and `addSubscriber()` check for **invalid inputs** (e.g., null or blank values) and **throw appropriate exceptions** (`IllegalArgumentException`, `NoSuchElementException`, `IllegalStateException`, etc.). Below are some scenarios where exceptions should be used:
- Invalid book details (title or author is blank/null) → `IllegalArgumentException`
- Removing a book that does not exist → `NoSuchElementException`
- Trying to remove a borrowed book → `IllegalStateException`
- Adding a subscriber with an invalid PIN → `ParseException`
- When throwing an exception, you can consider any *error message* that you find appropriate to describe the error as long as it **CONTAINS** the word “ERROR”. We won’t be checking for an exact match with these error messages. Blank error messages within thrown exceptions are **NOT** accepted in this assignment.
- Note that `ParseException` and `InstantiationException` are both checked exceptions defined by the Java API.

5. **File Handling for Books** The class supports saving and loading books from files:

- `saveBooks(File file)`: Saves the book inventory to a file in a `title:author` format.
- `loadBooks(File file)`: Reads book data from a file and loads it into the library. Incorrectly formatted lines are logged as errors but do not stop execution.

6. **Important Notes related to saveBooks method:**

- We recommend using the `PrintWriter` class defined in the `java.io` package to write and save a file.
- The `saveBooks` method saves the content of books `ArrayList` defined in the `ExceptionalLibrary` class in a file conforming to the following format for each line:
`title:author`
Do not save the books identifiers, or any information about their availability.
- The `saveBooks` method should NOT throw any exception. Any `FileNotFoundException` or `IOException` must be caught.
- Be sure to close the open resources using a **finally block** or a **try-with-resources block**.
- We highly recommend you to implement a private static tester method to check the correctness of your `saveBooks` method. Saved files can appear in your File Explorer and you can read them using any text editor.

7. **Important Notes related to loadBooks method:**

- The `loadBooks` method provides an interesting alternative to add books to the library inventory by loading a list of books saved in a File. These books will be added as new books to the `ExceptionalLibrary.books` `ArrayList`.
- We recommend using `java.util.Scanner` to read the input file.
- To process the format of each line, you can use the `String.split()` method
- **Format of the file to be loaded:** Each line (terminated by a newline character) in the file may either be blank, or contains the title of a book, followed by “:”, then followed by the author of the book.
There may be additional spaces between the title of a book (or the author of a book) and the punctuation.
All additional whitespaces at the beginning and at the end of the title or author names must be deleted using `String.trim()` method while processing the file.
If a line is badly formatted, it should be skipped and the method continues reading the next line.

- There are a few examples of text files that conform to this specification *books1.txt*, *books2.txt*, and **books3.txt** on the [P03 assignment page](#). We highly recommend you writing a private static tester method to check the correctness of your `loadBooks` method. This won't be graded by our autograder.
- While processing the file, `NullPointerException`, or `ArrayIndexOutOfBoundsException`, may be thrown if a line contains blank or only punctuation (":"), or no punctuation. Make sure to catch these exceptions within your `loadBooks` method and do not let them propagate to the calling method.
- If an `ArrayIndexOutOfBoundsException` or `NullPointerException` is thrown while processing a file line, skip that line and continue processing the next line if any.
- Do not forget to add a **finally block** to close open resources or use **try-with-resources block** while opening/reading the file.

3.2 ExceptionalLibraryTester

This class is dedicated to testing the static methods in the `ExceptionalLibrary` class to ensure they behave correctly in various scenarios. The testers should verify that:

- The system correctly manages books and subscribers.
- Books can be added, removed, borrowed, and returned properly.
- Exceptions are thrown and handled appropriately when invalid operations are attempted.

The required test methods that you must implement are provided in the starter code. You are highly encouraged to implement additional tester methods, especially to check the functioning of the *loadBooks* and *saveBooks* methods defined in the `ExceptionalLibrary` class.

Recommendation: To ensure consistent test results, call the `resetLibrarySystem()` method at the beginning of every tester method. This resets the list of books, subscribers, and the book ID generator, preventing data from previous tests from affecting subsequent test cases.

4 Assignment Submission

Congratulations on finishing this CS300 assignment! After verifying that your work is correct, and written clearly in a style that is consistent with the [CS300 Course Style Guide](#), you should submit your final work through [Gradescope](#). The only two (2) files that you must submit are

- `ExceptionalLibrary.java`
- `ExceptionalLibraryTester.java`

Your score for this assignment will be based on your “**active**” submission made prior to the assignment due date of Due: **9:59PM CT Thursday February 13**. The second portion of your grade for this assignment will be determined by running that same submission against additional offline automated grading tests after the submission deadline.

©Copyright: This write-up is a copyright programming assignment. It belongs to UW-Madison. This document should not be shared publicly beyond the CS300 instructors, CS300 Teaching Assistants, and CS300 Spring 2025 fellow students. Students are NOT also allowed to share the source code of their CS300 projects on any public site including GitHub, Bitbucket, etc.
--